

Figure 1: Output of plot2d()

```
(%o4)                2^4 3^2 5
(%i5) expand(6!);
(%o5)                720
(%i6) quit();
```

factor() and *expand()* are two complementary functions while doing factorisation and simplification, respectively. Note the way the powers in the output are printed. For example, 6! is factorised as $2^4 * 3^2 * 5$. In case you want the output on a single line, you can use the command *string()*, which we shall use from now on. Also, note the use of output labels as inputs to the commands, namely *expand(%o2)* in the above example.

Integration and differentiation

Octave has enabled you to do integration and differentiation, but more of definite integrals and differences. Yes, it has indefinite operations for polynomials. But with Maxima, you can just plug in the expressions, as you usually do in your math notebooks. This is what it means:

```
$ maxima -q
(%i1) integrate(cos(x),x);
(%o1)                sin(x)
(%i2) diff(cos(x),x);
(%o2)                - sin(x)
(%i3) string(integrate(1/(a^2 + x^2)^(3/2),x));
(%o3)                x/(a^2*sqrt(x^2+a^2))
(%i4) string(diff(log(x), x));
(%o4)                1/x
(%i5) y: (x+1)^2$ /* $ suppresses the output */
(%i6) string(diff(y, x));
(%o6)                2*(x+1)
(%i7) string(integrate(y, x));
(%o7)                x^3/3+x^2+x
(%i8) quit();
```

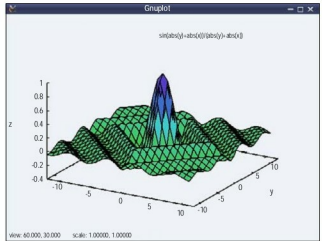


Figure 2: Output of plot3d()

Variable assignments can be done in Maxima using the colon (:), as shown in the input %i5, above. From there onwards, the variable can be used instead of the original expression, as shown in the inputs %i6 and %i7. Note that the variable assignment assigns the complete symbolic expression, unlike the usual value assignments.

Solving equations

Whether it is a linear set of equations, polynomial equations or, for that matter, non-linear equations, Maxima can solve most of them with ease, and even that, in an analytical way. Here are a few common examples: 1) The two solutions of a quadratic equation; 2) Two simultaneous equations in two variables; 3) A non-linear equation with irrational number e . A demonstration of the Maxima code follows:

```
$ maxima -q
(%i1) string(solve(a*x^2 + b*x + c = 0, x));
(%o1) [x = -(sqrt(b^2-4*a*c)+b)/(2*a), x = (sqrt(b^2-4*a*c)-b)/(2*a)]
(%i2) string(solve([a*x + b*y = c, d*x + e*y = f], [x, y]));
(%o2) [[x = -(c*e-b*f)/(b*d-a*e), y = (c*d-a*f)/(b*d-a*e)]]
(%i3) string(solve([exp(x) + 1/exp(x) = a], [x]));
(%o3) [x = log(a/2-sqrt(a^2-4)/2), x = log(sqrt(a^2-4)/2+a/2)]
(%i8) quit();
```

For ages, equation solving has been one of the most fascinating as well as challenging tasks while solving mathematical problems. So, when the computer is there to aid us, we do try to go beyond human limitations. We have just seen the well-known formula for the two solutions of a quadratic equation. How about the same for a cubic equation? It looks really complicated and, hence, we typically don't use it. But, if you still want to see it, here it is:

Continued on page 87...